

Rapport technique

PROTOCOLE DE CALIBRATION DE 3 CAMÉRAS 360°

ALEXANDRE JOLY

Sommaire

I)	Description du protocole de calibration	2
	Introduction et directives importantes.....	2
	Exécution de la calibration intrinsèque	3
	Exécution de la localisation des caméras.....	3
II)	Calibration des paramètres intrinsèques et de la distorsion focale	4
III)	Estimation des paramètres extrinsèques / localisation des caméras.....	5
	Résumé	5
	Partie Python	5
	Partie MATLAB – fonction LocateCameras.m	6
IV)	Fonctions de calibration.....	7
	Fonction CameraCalibration(nc,nr,path)	7
	Fonctions UndistortClassic et UndistortRemapping	8
V)	Utilisation de l’application MATLAB Camera Calibrator	9

I) Description du protocole de calibration

Introduction et directives importantes

Pour plus de simplicité, l'ensemble du code est exécutable via un unique fichier « main.py ». Cependant, les deux parties du protocole (calibration des paramètres intrinsèques et localisation des caméras) sont écrites dans deux fichiers distincts, pour plus de malléabilité. Les deux peuvent donc être exécutés séparément et indépendamment l'un de l'autre.

Selon l'emplacement du dossier contenant l'ensemble du code écrit pour ce projet, il faudra adapter le nom des chemins vers les images dans les fichiers « insta360pro2_IntrinsicsAndDistortion.py » et « insta360pro2_Extrinsics.py » :

```
14 # Define paths
15 path1 = "C:/Users/AT10820/Desktop/Insta360Pro2Calibration/Pictures/cam1/location/"
16 path2 = "C:/Users/AT10820/Desktop/Insta360Pro2Calibration/Pictures/cam2/location/"
17 path3 = "C:/Users/AT10820/Desktop/Insta360Pro2Calibration/Pictures/cam3/location/"
```

Chemins définis dans insta360pro2_Extrinsics.py

Aussi, selon le damier utilisé pour ce protocole de calibration, il faudra adapter des paramètres dans les scripts. Le fichier « insta360pro2_IntrinsicsAndDistortion.py » a besoin de nc et nr, les nombres de colonnes et de lignes à détecter sur le damier. Attention, si le damier fait 8x12 carreaux par exemple, il faut indiquer nc = 7 et nr = 11 car la détection se fait sur les carreaux internes du patron :

```
20 # Define pattern parameters
21 nc,nr = 7,11
```

De la même façon, le fichier « insta360pro2_Extrinsics.py » a besoin de la longueur d'un carreau sur le damier en millimètres (ici 148 mm) :

```
22 # Define pattern parameters
23 squareSizeInMM = 148.0
```

Exécution de la calibration intrinsèque

Lors de cette partie, le code va se connecter au live origin 1 de la première caméra. Il s'agit de la vue fisheye du capteur #1 de cette caméra. L'utilisateur peut choisir une autre vue de la caméra en appuyant sur la touche « N » (pour Next) afin de passer à la vue suivante. Attention cependant : il n'y a pas de retour en arrière possible, donc il est conseillé de s'en tenir à une vue et adapter légèrement la position de la caméra pour les photos. Une fois dans la vue qu'il souhaite, l'utilisateur capture des photos de calibration (avec la touche « C ») sur lesquelles il met en évidence le damier de calibration à différentes positions et sous différents angles afin de fournir un maximum d'information à l'algorithme. Plus de photos sont prises, plus les résultats seront précis (une dizaine suffit).

Une fois que l'utilisateur a pris toutes les photos de calibrations qu'il souhaite sur une vue de la première caméra, il appuie sur la touche « N » autant de fois qu'il faut pour passer toutes les vues qu'il reste sur cette caméra, afin de passer à la caméra suivante, puis répète les mêmes opérations.

Lorsque toutes les photos de calibrations sont prises et que l'utilisateur appuie sur « N » pour la dernière fois en passant la vue 6 de la troisième caméra, le script se déconnecte du dernier live stream et lance la calibration. À partir de là, il n'y a plus rien à faire à part observer la console, dans laquelle les résultats s'affichent : les matrices intrinsèques et coefficients de distorsion des 3 caméras. L'étape de calibration des paramètres intrinsèques est terminée.

Exécution de la localisation des caméras

Pour localiser les caméras les unes par rapport aux autres, l'idée est simple : placer un damier dans le champ de vision des 3 caméras et capturer la vision d'un capteur de chacune d'entre elles, où le damier est entièrement visible. L'algorithme fait en sorte de calculer les vecteurs de translation et les matrices de rotation du damier par rapport à chaque caméra, et fait ensuite la reprojection inverse afin d'avoir les coordonnées des caméras dans un même repère.

L'exécution de cette partie de la calibration est donc à peu près la même que dans la partie précédente : l'utilisateur passe d'une vue à la suivante avec la touche « N » et capture des photos avec la touche « C ». Seule différence ici : il ne doit prendre qu'une seule photo pour chaque caméra. À la fin de la partie de capture d'images, il doit avoir un total de 3 photos.

Le reste est mené par le code seul, qui appelle MATLAB et affiche une visualisation de la localisation des caméras les unes par rapport aux autres dans le repère du damier.

II) Calibration des paramètres intrinsèques et de la distorsion focale (fichier : insta360pro2_IntrinsicsAndDistortion.py)

On définit d'abord les différentes constantes qui nous seront utiles :

- Les chemins des dossiers où enregistrer/consulter nos photos de calibration
- Un tableau contenant ces chemins (utile pour les boucles)
- Les dimensions du patron qu'on utilise pour la calibration : nombre de colonnes (nc) et nombre de lignes (nr)
- 3 tableaux contenant les adresses des streams auxquels se connecter, ainsi qu'un tableau les contenant tous (ici encore, utile pour les boucles)

Ensuite, on lance les lives « origin » des 3 caméras via le script **LiveOriginMulti.py**. Ces lives permettent de visualiser chaque senseur d'une même caméra, numéroté de 1 à 6.

Vient ensuite la boucle principale du code, lors de laquelle on va capturer les images de calibration. On commence par initialiser nos indexes, puis on réalise la même suite d'opérations pour chaque caméra :

On affiche sur quelle caméra on est connecté.

On définit le tableau d'adresses de streams correspondant (vues 1 à 6).

On lance ensuite la capture des 6 streams et on initialise l'index de la 1^{ère} image qu'on capturera à 1.

Puis, pour chaque vue, on capture le stream correspondant image par image.

Avant d'afficher une frame, on lui fait faire d'abord une rotation de 90 degrés dans le sens horaire car les streams origin sont par défaut pivotés de façon à être horizontaux. On réalise aussi un redimensionnement de l'image pour qu'elle tienne entièrement sur notre écran (dans notre cas, on se met en HD : 1080x1920). Enfin, si l'image n'est pas vide (None), on l'affiche et l'utilisateur peut choisir de capturer une frame avec la touche (C) ou bien de passer au stream suivant avec la touche (N) du clavier. La capture d'une image entraîne directement sa sauvegarde dans le dossier correspondant à la caméra à laquelle on est actuellement connecté.

Un message en console indique lorsqu'une photo est capturée et sauvegardée, et un autre indique aussi si on passe à la vue suivante d'une même caméra, ou si on passe à la caméra suivante une fois les 6 vues passées. Au moment de passer d'un stream à un autre, on libère l'objet de capture vidéo du stream correspondant et on ferme la fenêtre qu'on avait construit pour l'afficher.

À chaque passage à la caméra suivante, l'index de caméra (cam) est actualisé, ainsi que celui des lives (idx).

Une fois qu'on a vu les 6 lives des 3 caméras et qu'on a fermé le tout dernier, l'étape de calibration commence.

On fait une boucle for pour réaliser les mêmes opérations pour chaque caméra :

On définit une chaîne de caractère « files » qui pointe vers tous les fichiers de type JPEG contenus dans le dossier où on a enregistré les images de cette caméra, grâce à une concaténation entre le chemin du dossier et la chaîne « *.jpg ».

On crée ensuite un objet « images » qui vient lire les noms complets de chaque image (en comptant le chemin qui y amène).

Puis, on utilise la fonction `CameraCalibration` qui renvoie la matrice intrinsèque et le vecteur de distorsion. La fonction renvoie aussi beaucoup d'autres sorties, mais celles-ci ne nous intéressent pas ici. Voir l'explication des fonctions de calibration via **`calib_functions.py`**.

Enfin, on enregistre la matrice intrinsèque et le vecteur contenant les coefficients de distorsion focale pour pouvoir les utiliser plus tard. Dans notre exemple, on ne fait que les afficher, mais il est donc possible d'utiliser ces paramètres si jamais des modifications sont faites sur ce code par des tiers.

III) Estimation des paramètres extrinsèques / localisation des caméras (fichier : `insta360pro2_Extrinsics.py`)

Résumé

Ce script permet la localisation des 3 caméras Insta360 Pro2 les unes par rapport aux autres. Pour ce faire, on place d'abord un patron type damier dans le champ de vision des 3 caméras, et, pour chacune d'elle, on capture l'image d'un senseur où le patron est visible. Ensuite, le script se connecte à une session MATLAB pour déterminer la localisation des caméras et les visualiser grâce à la fonction **`LocateCameras.m`**

Partie Python

Comme pour l'estimation des paramètres intrinsèques, on commence par définir quelques constantes : chemins vers les dossiers où les images seront enregistrées et ensuite consultées, taille d'un carreau du patron et adresses des lives origin des 3 caméras. On s'occupe aussi de supprimer les images qu'on avait capturées lors de la précédente utilisation du script, pour ne pas avoir à le faire à la main à chaque fois.

Ensuite, on lance les lives des 6 senseurs de chaque caméra via **`LiveOriginMulti.py`** et commence la boucle principale où l'utilisateur va pouvoir capturer ses images. Tout est réalisé exactement de la même façon que pour la capture des images de calibration : cf. **`insta360pro2_IntrinsicsAndDistortion.py`**

La méthode de capture est la suivante : l'utilisateur utilise la touche (N) pour passer à la vue suivante. Une fois qu'il se trouve sur la vue dans laquelle le damier se trouve, il utilise la touche (C) pour capturer l'image, puis passe à la caméra suivante en passant toutes les vues restantes (N). Et l'opération est à répéter pour chaque caméra, donc un total de 3 fois.

Une fois les images capturées, le script lance une session MATLAB. Pour cela, il a été nécessaire d'installer le **MATLAB Engine API for Python** qui permet d'appeler des fonctions écrites en MATLAB depuis un script écrit en Python. Voici la documentation nécessaire pour l'installation et l'utilisation de ce package :

https://www.mathworks.com/help/matlab/matlab_external/install-the-matlab-engine-for-python.html

https://www.mathworks.com/help/matlab/matlab_external/get-started-with-matlab-engine-for-python.html

Grâce à ça, on peut appeler la fonction **LocateCameras.m** qui permet de localiser les caméras les unes par rapport aux autres.

Partie MATLAB – fonction **LocateCameras.m**

Cette fonction prend en entrée les noms complets des 3 images prises pour la localisation, ainsi que la taille d'un carreau du damier en millimètres.

Pour commencer, on stocke les noms des fichiers dans une cellule qu'on appelle « **imageFilesNames** ». Puis, on réalise la détection du damier de calibration sur les images. Pour cela, on initialise un détecteur spécifique aux patrons du type damier que l'on utilise, et on détermine les points images avec la fonction **detectPatternPoints** de MATLAB, en lui précisant bien que la distorsion est forte sur les images (fisheye oblige). Cette fonction renvoie aussi la variable « **imagesUsed** » qui indique par un 1 les images sur lesquelles un damier a été détecté et un 0 celles où il n'a pas été détecté.

Ceci nous permet alors de réduire **imageFilesNames** uniquement aux images sur lesquelles le damier a pu être détecté, pour ne pas produire d'erreur.

Ensuite, on fait en sorte de récupérer la taille des images en ouvrant la première de la liste grâce à un simple **imread(filename)** et **size(image)**. Cette étape nous est utile pour la calibration des images par MATLAB. Mais avant cela, il faut générer les coordonnées des coins dans l'espace 3D connaissant la taille des carreaux. C'est ce que fait la fonction **generateWorldPoints** de MATLAB qui, à partir de l'initialisation du détecteur et de la taille d'un carreau renvoie un tableau des coordonnées de chaque point sous la forme (0,0),(0,148),(0,296)...(1480,1036), ici dans le cas de notre étude où notre damier fait 8x12 carreaux de 148 mm chacun.

La calibration peut enfin être réalisée : on utilise la fonction **estimateCameraParameters** de MATLAB en lui fournissant les points images et les points **worldPoints** générés précédemment. On indique aussi à la fonction de ne pas estimer la distorsion tangentielle qui ne nous intéresse pas dans notre cas. On fixe aussi le nombre de coefficients de distorsion radiale à 3, l'unité aux millimètres, ainsi que la dimension des images grâce au procédé réalisé plus tôt.

Cette étape a pour effet de créer un objet contenant les paramètres de la caméra qui a pris les photos. ATTENTION : ici, on utilise cette fonction uniquement pour les paramètres extrinsèques ! Les paramètres intrinsèques renvoyés ne sont absolument pas pertinents car les 3 photos utilisées ont été prises avec 3 objectifs différents.

La fonction affiche ensuite deux choses :

- La moyenne d'erreurs de reprojections en pixels pour chaque image considérée (fonction `showReprojectionErrors()`)
- La visualisation des positions et rotations des caméras par rapport au damier (fonction `showExtrinsicsV2()`) IMPORTANT : cette fonction est dérivée de `showExtrinsics.m`, fonction de MATLAB que j'ai modifiée afin qu'elle renvoie les vecteurs de translation du centre des caméras par rapport au damier.

Par la suite, on récupère aussi les matrices de translation et on fait tout une étape d'affichage directement depuis MATLAB.

Les vecteurs de translation et les matrices de rotation sont tous envoyés en sortie de cette fonction.

À la fin, une condition sur la valeur de retour de la fonction **`waitforbuttonpress`** permet de bloquer la fonction et donc de garder les figures ouvertes tant que l'utilisateur n'appuie pas sur une touche du clavier. Dès qu'il le fera, la fonction se débloquent et le script Python pourra s'arrêter.

IV) Fonctions de calibration (fichier : `calib_functions.py`)

Pour réaliser la calibration des caméras, j'ai développé quelques fonctions utiles à cet effet, largement inspirées du tutoriel open-cv accessible au lien ci-dessous :

https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

Fonction `CameraCalibration(nc,nr,path)`

Cette fonction prend en argument le nombre de colonnes et de lignes du damier à détecter ainsi que le chemin vers les photos de calibration à considérer.

On commence par initialiser les points objet sous la forme $(0,0,0)$, $(1,0,0)$... $(nc,nr,0)$, désignant les coordonnées de chaque coin à détecter par la suite.

On initialise ensuite les tableaux qui contiendront les points objet (3D) et les points image (2D) en les laissant vides. Puis, on lit toutes les images enregistrées dans le dossier indiqué par le chemin donné en argument de la fonction. On initialise alors le nombre total d'images dans ce dossier à 0, même chose pour le nombre d'images où le damier a été correctement détecté.

Commence alors la recherche des coins : dans une boucle, pour chaque fichier du dossier, on vient lire l'image correspondante et on la transforme pour la passer de couleur à niveaux de gris. Ceci permet de détecter correctement les coins du damier, que l'on réalise juste après avec la fonction d'open-cv **`findChessboardCorners`**, qui renvoie les coordonnées des coins détectés ainsi qu'un booléen qui indique si le damier a bien été détecté ou non.

Par la suite, si ce booléen vaut « True », on incrémente le nombre d'images correctement analysées et on vient ajouter le point objet correspondant dans son tableau, ainsi que les coordonnées du coin détecté dans le tableau des points images. On fait aussi en sorte d'incrémenter le nombre total d'images à chaque passage de la boucle.

On affiche ensuite un message indiquant le nombre de photos correctement détectées et le nombre total d'image qu'il y avait dans le dossier.

L'étape de calibration peut alors commencer : on utilise la fonction d'open-cv **calibrateCamera** qui prend en arguments les points objets et les points image afin de renvoyer la matrice intrinsèque, le vecteur contenant les coefficients de distorsion radiale ainsi que les vecteurs de rotation et de translation correspondants.

Enfin, on vient affiner la matrice intrinsèque en utilisant la fonction **getOptimalNewCameraMatrix** qui prend en compte la distorsion et donne une matrice plus proche de la réalité.

La fonction renvoie à la fin tous les paramètres calculés : matrice intrinsèque de base, matrice intrinsèque affinée, coefficients de distorsion, largeur et hauteur des images, le ROI*, les points images, les points objet et les vecteurs de rotation et de translation. (*le ROI est un paramètre qui sert pour la correction de la distorsion des images)

Fonctions **UndistortClassic** et **UndistortRemapping**

La bibliothèque open-cv propose deux solutions pour la correction de la distorsion radiale sur une image.

La première, la plus classique, est illustrée dans la fonction **UndistortClassic**. À partir de l'objet « images » donnée en argument (qui contient l'ensemble des noms de fichiers de type JPG dans le dossier voulu), on vient réaliser les mêmes opérations pour chaque image :

- On crée l'image non déformée grâce à la fonction **undistort** d'open-cv.
- On rogne l'image obtenue à partir du ROI calculé précédemment dans l'étape de calibration.
- On sauvegarde l'image dans un dossier voulu, donné en argument de la fonction.

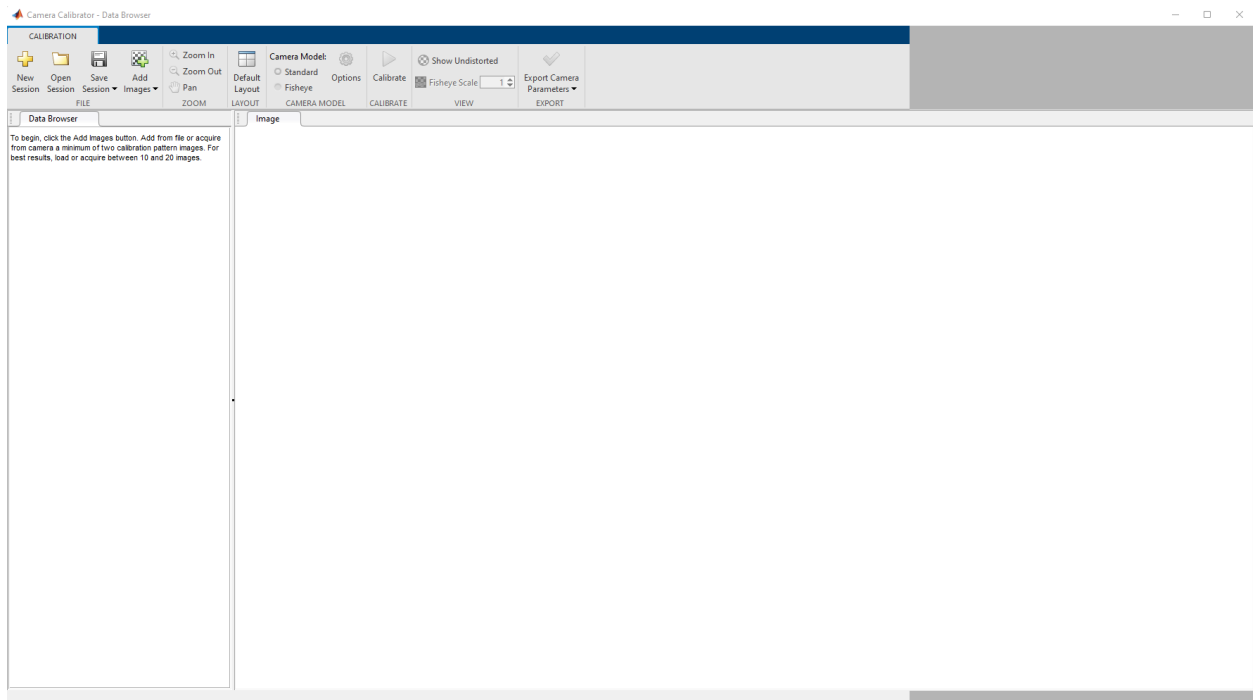
La deuxième solution retire la distorsion en utilisant du remapping. Pour chaque image :

- On crée deux objets **mapx** et **mapy** avec la fonction **initUndistortRectifyMap** à qui on donne la matrice intrinsèque, les coefficients de distorsion, la matrice intrinsèque affinée, ainsi que la largeur et hauteur de l'image.
- On crée l'image non déformée à partir de mapx et mapy et de l'image initiale en utilisant **remap**, là encore une fonction open-cv.
- On rogne l'image obtenue à partir du ROI calculé précédemment dans l'étape de calibration.
- On sauvegarde l'image dans un dossier voulu, donné en argument de la fonction.

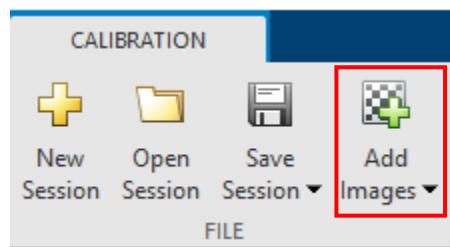
V) Utilisation de l'application MATLAB Camera Calibrator

Le logiciel MATLAB regorge de ressources et d'outils dans bien des domaines différents. En plus de nombreuses fonctions pour la calibration de caméra, il propose entre autres une application à part entière dédiée à cet effet : l'application **Camera Calibrator**.

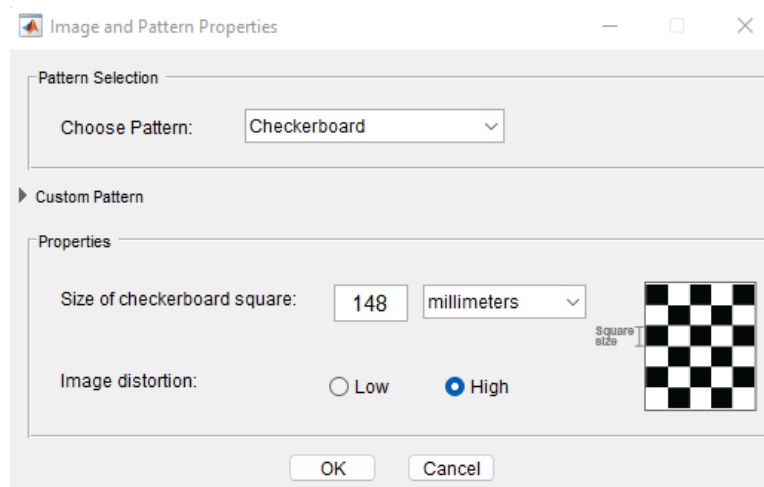
À l'ouverture, voici comment se présente l'outil :



Et pour l'utiliser, rien de plus simple ! Il suffit d'ouvrir les photos de calibrations désirées en cliquant sur **Add Images** :



Lorsque les photos sont sélectionnées, une fenêtre s'ouvre et demande des précisions sur les propriétés du damier à détecter sur les images de calibrations :

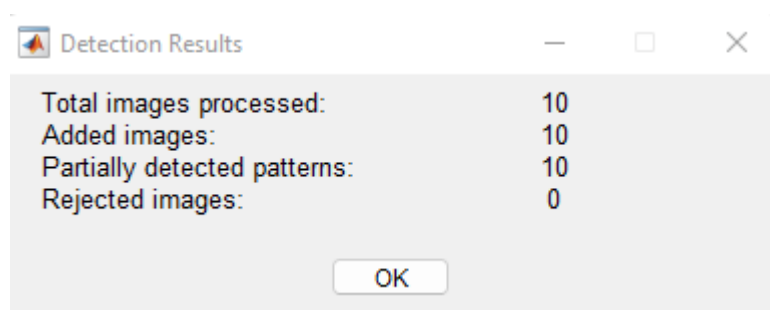


On a alors la possibilité d'indiquer si le patron à détecter est de type **Checkerboard**, **Asymmetric Circle Grid** ou **Symmetric Circle Grid**.

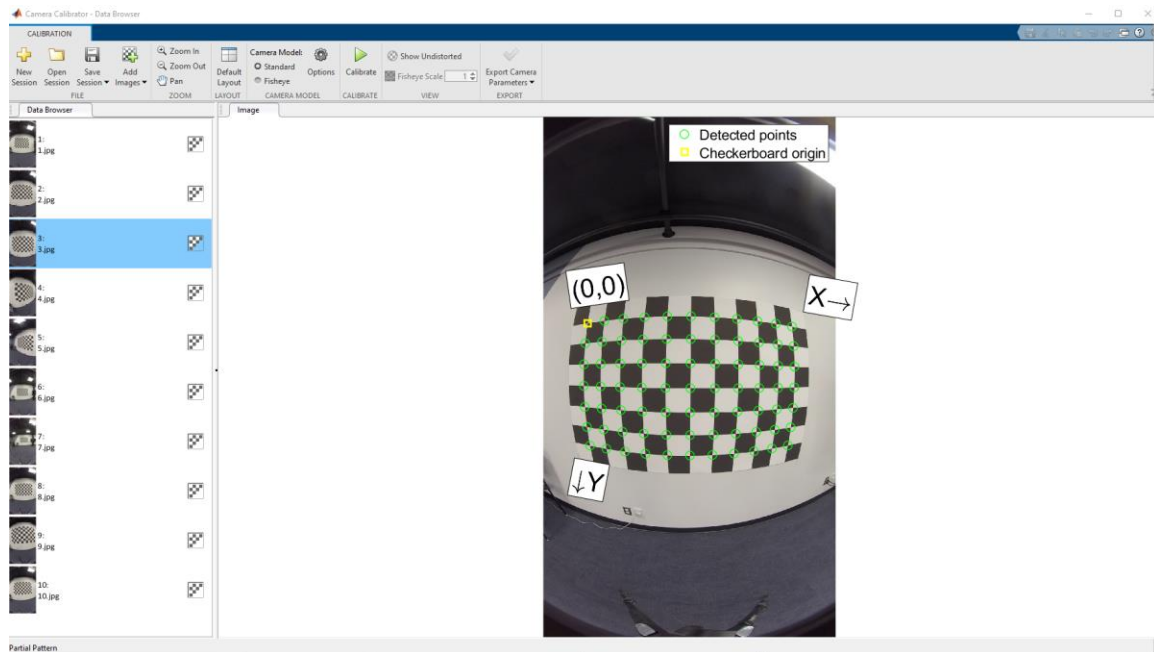
Dans notre cas, nous utilisons un damier simple, donc de type **Checkerboard**.

On finit donc par indiquer la taille d'un carreau sur le damier, en précisant l'unité, ainsi que le type de distorsion : élevé ou bas. Avec le damier utilisé lors de notre projet, les paramètres à entrer sont tels qu'indiqués sur la capture d'écran ci-dessus.

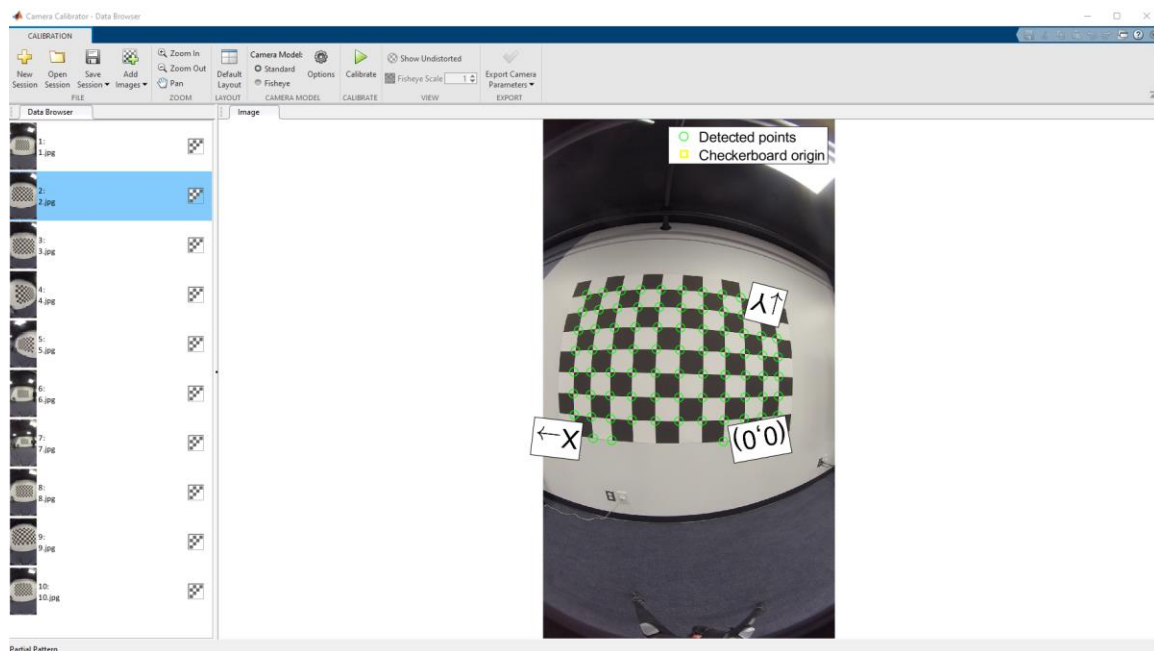
Une fois la configuration du pattern faite, cliquer **OK** et une fenêtre de résumé s'affiche pour indiquer le nombre total d'images considérées :



Une fois cette étape réalisée, un chargement se produit le temps que toutes les images soient analysées par l'application. C'est l'étape de détection des coins du damier, car après cela, l'application permet une visualisation de tous les points détectés ainsi que l'origine considérée du damier :

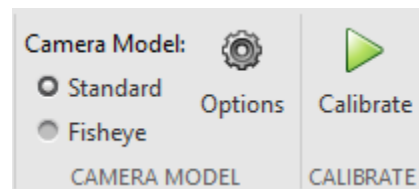


Cette origine est désignée tout en haut à gauche lorsque le damier est entièrement détecté. Cependant, il arrive que le damier ne soit pas détecté parfaitement, ou de la même façon, et l'origine peut se retrouver ailleurs, la plupart du temps dans le coin en bas à droite, comme c'est le cas dans l'exemple ci-dessous :



Bien qu'il ne pose pas vraiment de problème pour l'estimation des paramètres intrinsèques de la caméra utilisée pour prendre les images de calibration, cela a en revanche une très grande incidence sur la détermination des paramètres extrinsèques : pour les images où l'origine du damier est en bas à droite, comme l'origine a été retournée, la rotation et la translation obtenues en sont aussi retournées par rapport à ce qu'elles devraient être ! Attention donc à ce que les points d'origines soient bien tous déterminés au même endroit pour toutes vos photos si votre intérêt est d'estimer les paramètres extrinsèques et de faire de la localisation.

Par la suite, il est possible de configurer des paramètres :



Comme on peut le voir sur l'image ci-dessus, il est possible d'indiquer si on veut déterminer des paramètres de caméra standard ou de caméra fisheye. Certains des paramètres obtenus seront différents, comme il s'agit de deux modèles différents.

Le bouton **Options** permet d'indiquer si on veut 2 ou 3 coefficients de distorsion radiale, et si on veut calculer aussi l'erreur et la distorsion tangentielle. Il y a aussi un bouton **OptimizationOptions** dans ce menu, qui permet de donner une initialisation de la matrice intrinsèque et/ou des coefficients de distorsion radiale, si on les connaît, afin d'aider l'application à déterminer les bons paramètres.

Une fois toutes ces configurations réalisées, il ne reste plus qu'à cliquer sur **Calibrate**.

Après un bref instant de calcul, l'application a calculé tous les paramètres de la caméra, intrinsèques comme extrinsèques. Elle permet une visualisation de l'erreur moyenne de calcul sur chaque image (en pixels) ainsi qu'une visualisation des extrinsèques, en caméra-centrique ou pattern-centrique (selon si on veut visualiser les positions des patterns dans le repère de la caméra, ou bien les positions de la caméra dans le repère du pattern).

Une fois la calibration réalisée, il est possible de visualiser les images dont la déformation a été corrigée, en cliquant sur le bouton **Show Undistorted**, toujours dans la barre d'outils de l'application.

Enfin, il est possible d'**exporter** les résultats obtenus en cliquant sur **Export Camera Parameters**. Là, deux options s'offrent à l'utilisateur : il peut exporter les paramètres de la caméra sous la forme d'un objet MATLAB qui les contient tous, et il peut aussi cliquer sur **Generate MATLAB Script** qui génère le code MATLAB permettant l'ouverture des photos utilisées, l'estimation des paramètres et la visualisation des erreurs moyennes et des paramètres extrinsèques, tout comme ce que l'application fait mais sans avoir à passer par elle.